

DEVALIVE

Quality Assurance & Deployment Document (QADD)

Version: 1.0

Document Status: Approved

Purpose:

This document defines the testing, quality assurance, deployment, production release, environment management, and maintenance standards for DevAlive.

The objective is to ensure that every release is:

- Stable
- Secure
- Reliable
- Production Ready

1. Quality Assurance Philosophy

DevAlive must follow:

``text id="6s8zxa" Build → Test → Deploy

Never :

``text id="cz2vzx"
Build → Deploy

All critical functionality must be verified before production deployment.

2. Quality Assurance Objectives

The QA process must ensure:

Authentication Works

Monitoring Works

Notifications Work

Analytics Are Accurate

APIs Are Stable

UI Is Responsive

Security Rules Are Enforced

3. Testing Strategy Overview

DevAlive uses multiple testing layers.

Unit Testing

Integration Testing

API Testing

Frontend Testing

Manual QA Testing

Production Verification

4. Unit Testing Strategy

Purpose:

Verify individual functions.

Examples:

```text id="7j5kft" Password Hashing

JWT Generation

Uptime Calculation

Response Time Calculation

Notification Logic

---

Expected Result:

```text id="7cws1j"  
Every function behaves correctly in isolation.

5. Integration Testing Strategy

Purpose:

Verify communication between modules.

Examples:

```text id="wgstjz" Auth Service ↔ Database

Projects ↔ Monitoring Engine

Monitoring Engine ↔ Notifications

Analytics ↔ Monitoring Logs

---

Expected Result:

```
```text id="7z42cs"
System modules work together correctly.
```

6. API Testing Strategy

Purpose:

Verify all endpoints.

Authentication APIs

```
```text id="g1vulj" Register
```

Login

Current User

```

```

Projects APIs

```
```text id="sqsrdf"
Create Project
```

Update Project

Delete Project

Get Projects

Monitoring APIs

```
```text id="4o2z7e" Logs
```

History

Manual Check

---

Analytics APIs

```text id="3l7gxg"

Dashboard

Project Analytics

Expected Verification

```text id="l1b5e6" Status Codes

Validation

Authorization

Response Structure

---

# 7. Frontend Testing Strategy

Purpose:

Verify UI behavior.

---

Test Areas

```text id="khr7p9"

Landing Page

Authentication

Dashboard

Projects

Analytics

Logs

Notifications

Settings

Verify

```text id="u8m1v8" Rendering

Navigation

Form Validation

API Integration

Responsiveness

---

# 8. Monitoring Engine Testing

Critical Module

Highest Priority

---

Test Cases

```text id="j0n8l7"

Healthy Endpoint

Offline Endpoint

500 Error

404 Error

Timeout

Slow Response

Verify

```text id="a9s6cf" Status Detection

Statistics Update

Log Creation

Notification Triggering

---

## # 9. Notification Testing

Test Cases

```text id="3kz55n"

Downtime Alert

Recovery Alert

Monitoring Failure

Verify

```text id="eb84q5" Notification Created

Email Sent

Database Record Stored

---

## # 10. Security Testing

Verify:

```text id="d9h0r4"

JWT Validation

Protected Routes

Password Encryption

Rate Limiting

Input Validation

Authorization Rules

Security Tests

```text id="crv0al" Unauthorized Requests

Invalid JWT

Invalid ObjectId

Malformed Inputs

---

### # 11. User Acceptance Testing (UAT)

Purpose:

Verify system from user perspective.

---

Scenario

```text id="hwrdfx"

Register

↓

Login

↓

Add Project

↓

Monitoring Starts

↓

View Dashboard

↓

Receive Alert

Expected Result

```text id="l7zt8v" Complete user journey works without issues.

---

## # 12. Bug Classification

---

Critical

Examples

```text id="c58d2i"

Login Failure

Monitoring Not Running

Database Failure

Must Fix Before Release.

Major

Examples

```text id="v6hz0m" Incorrect Analytics

Broken Dashboard Widgets

Must Fix Before Release.

---

Minor

Examples

```text id="7q4c4x"

UI Alignment Issues

Typos

Can Be Fixed Later.

13. Deployment Architecture

Environment Structure

```text id="5hh9c6" Local Development

Staging Environment

Production Environment

```

14. Frontend Deployment

Platform

```text id="x5a2z4"  
Vercel
```

Responsibilities

```text id="42mnj2" React Hosting

CDN Delivery

Automatic Deployments

```

15. Backend Deployment

Platform

```text id="2skq1s"  
Render
```

Responsibilities

```text id="r8r1yo" API Hosting

Monitoring Engine

## Scheduler Execution

```

Future Scale

```text id="wdgh5u"  
Railway  
  
AWS  
  
DigitalOcean
```

16. Database Deployment

Platform

```
```text id="n8i2vm" MongoDB Atlas
```

```

Responsibilities

```text id="8skpj3"  
User Data  
  
Projects  
  
Monitoring Logs  
  
Notifications  
  
Analytics Data
```

17. Environment Variables

Backend

```
```text id="fuj5k8" PORT
```

MONGODB\_URI

JWT\_SECRET

SMTP\_HOST

SMTP\_PORT

SMTP\_USER

SMTP\_PASSWORD

---

Frontend

```
```text id="yykhkl"
VITE_API_URL
```

Rules

```text id="ngnq3w" Never Commit Secrets

Never Hardcode Credentials

---

# 18. Production Configuration

Backend

```
```text id="2qqh1h"
NODE_ENV=production
```

Frontend

```text id="95z6ul" Production API URL

---

Database

```
```text id="zyv7mz"
Production Cluster
```

19. CI/CD Strategy

Current MVP

```
```text id="9rksdc" GitHub
```

Vercel Auto Deploy

Render Auto Deploy

```

```

Workflow

```
```text id="s7cf7i"
Push To GitHub
↓
Build Triggered
↓
Tests Executed
↓
Deployment Started
↓
Production Updated
```

Future

```
```text id="4jggaj" GitHub Actions
```

Automated Test Pipeline

```

```

```
20. Production Readiness Checklist
```

Authentication

```
```text id="jvt2y7"
Verified
```

Authorization

```
```text id="e6gd6q" Verified
```

---

Monitoring Engine

```
```text id="bsiqkz"
Verified
```

Notifications

```
```text id="g9prxv" Verified
```

---

Analytics

```
```text id="8m3c2l"
Verified
```

Responsive Design

```
```text id="pvyd97" Verified
```

---

Environment Variables

```
```text id="78a18k"
Configured
```

Security Headers

```text id="m3mzuy" Enabled

---

Rate Limiting

```text id="8sm6b6"  
Enabled

Error Handling

```text id="mrp99t" Verified

---

# 21. Backup & Recovery Strategy

Database

```text id="qj8z2q"  
MongoDB Atlas Backups

Logs

```text id="lvxth0" Monitoring Logs Stored

---

Recovery Goal

```text id="3a2y34"  
Minimal Data Loss

22. Post Deployment Monitoring

Monitor

```text id="w0t5uy" Backend Health

Database Health

Monitoring Scheduler

Email Delivery

API Performance

---

Tools (Future)

```text id="z7u1uh"

Sentry

BetterStack

UptimeRobot

23. Quality Assurance & Deployment Summary

DevAlive follows a production-oriented QA and deployment process that emphasizes:

- Reliability
- Security
- Stability
- Automated Deployment
- Monitoring Validation

The platform must pass testing, verification, and production-readiness checks before every release.

This ensures that DevAlive delivers a dependable SaaS experience for developers and future customers.